C04 ASSESSMENT TOOL 3(Comparative Analysis)

Name: Y.Thanushma

Registration Number: 192425355

Course Code: CSA0617-Design & Analysis of Algorithms

1. Coin Change vs Knapsack — DP Table Comparison

A. Coin Change Problem

Given: Coins = [1, 2, 5], Amount = 5

Here, dp[i][j] = number of ways to make amount j using the first i coins.

Transition:

- If coin > amount: **dp[i][j] = dp[i-1][j]**
- Otherwise:
  dp[i][j] = dp[i-1][j] + dp[i][j-coin]

The first term excludes the coin, while the second includes it and allows the same coin again.

Coin Change DP Table

Coins \ Amount	0	1	2	3	4	5
0	1	0	0	0	0	0
1	1	1	1	1	1	1
1,2	1	1	2	2	3	3
1,2,5	1	1	2	2	3	4

So, the number of ways to make amount 5 is 4:

1. 1+1+1+1+1
2. 1+1+1+2
3. 1+2+2

4. 5

B. 0/1 Knapsack Problem

Given:

- Weights = [2,3,4]
- Values = [3,4,5]
- Capacity = 5

Here, dp[i][w] = maximum value that can be obtained using the first i items with capacity w.

Transition:

- If weight > capacity:
  dp[i][w] = dp[i-1][w]
- Otherwise:
  dp[i][w] = max(dp[i-1][w], value + dp[i-1][w-weight])

Notice that the second term uses i-1, because each item can be used only once.

Knapsack DP Table

Items \ Capacity	**0**	**1**	**2**	**3**	**4**	**5**
0	0	0	0	0	0	0
(2,3)	0	0	3	3	3	3
(2,3),(3,4)	0	0	3	4	4	7
(2,3),(3,4),(4,5)	0	0	3	4	5	7

The maximum value is 7, obtained by selecting:

- Weight 2, Value 3
- Weight 3, Value 4

Total weight = 5

Total value = 7

C. Comparison of Transitions

Feature	Coin Change	0/1 Knapsack
Goal	Count number of ways	Maximize value
Repetition	Coins can be reused	Items used once
Main transition	dp[i-1][j] + dp[i][j-coin]	max(dp[i-1][w], v + dp[i-1][w-wt])
Include state	Same row i	Previous row i-1
DP meaning	Number of combinations	Maximum value
Example result	4 ways	Maximum value = 7

Key Structural Difference

Coin Change:

dp[i][j] = dp[i-1][j] + dp[i][j-coin]

The same row is used when including a coin, allowing unlimited reuse.

Knapsack:

dp[i][w] = max(dp[i-1][w], v + dp[i-1][w-wt])

The previous row is used when including an item, preventing reuse.

D. Complexity

For Coin Change with n coins and amount A:

- Time: O(n × A)
- Space: O(n × A) for a 2D table
- Can be optimized to O(A) space.

For 0/1 Knapsack with n items and capacity W:

- Time: O(n × W)
- Space: O(n × W) for a 2D table

- Can be optimized to O(W) space.

Conclusion

Both problems use dynamic programming and have similar O(n × capacity) time complexity. The major structural difference is that Coin Change allows repeated use of a coin, while 0/1 Knapsack allows each item only once. This difference is reflected directly in their DP transitions.

2.DICE THROW VS LPS (DP STATE DEFINITION)

A. Dice Throw Problem

Given:

• Number of dice = 2

• Each die has 6 faces

• Required sum = 5

DP State:

Let dp[i][j] represent the number of ways to obtain sum j using i dice.

DP Table:

Dice \ Sum	**0**	**1**	**2**	**3**	**4**	**5**
0	1	0	0	0	0	0
1	0	1	1	1	1	1
2	0	0	1	2	3	4

For 2 dice and sum 5, the possible combinations are:

(1,4) (2,3) (3,2) (4,1)

Therefore, the number of solutions = 4.

Recurrence Relation:

dp[i][j] = Σ dp[i-1][j-k], where k = 1 to 6

Here, k represents the value obtained from the current die.

B. Longest Palindromic Subsequence (LPS)

Given String: ABCBA

DP State: Let dp[i][j] represent the length of the longest palindromic subsequence between index i and j.

DP Table:

i \ j	**A**	**B**	**C**	**B**	**A**
A	1	1	1	3	5
B		1	1	3	3
C			1	1	1
B				1	1
A					1

The complete string "ABCBA" is itself a palindrome.

Therefore:

LPS = ABCBA

Length of LPS = 5

Recurrence Relations:

If the characters at both ends are equal: dp[i][j] = dp[i+1][j-1] + 2

If the characters are different: dp[i][j] = max(dp[i+1][j], dp[i][j-1])

For "ABCBA": dp[0][4] = dp[1][3] + 2

= 3 + 2 = 5

C. Comparison of DP States and Transitions

Feature	**Dice Throw**	**LPS**
Input	Dice and target sum	String
DP State	dp[dice][sum]	dp[i][j]

Goal	Count number of ways	Find longest palindrome
State Meaning	Ways to obtain a particular sum	LPS length between two indices
Transition	Sum over possible dice values	Compare two characters
Result	4 ways	Length = 5
DP Type	Counting DP	Optimization DP

Main Difference:

Dice Throw: The state depends on the number of dice and the current sum.

$dp[d][s] = \Sigma\ dp[d-1][s-k]$

It counts all possible ways to obtain the required sum.

LPS: The state depends on the starting and ending positions of the string.

dp[i][j] . It finds the maximum-length palindromic subsequence.

D. Complexity Analysis

Dice Throw:

For D dice and target sum S: Time Complexity = $O(D \times S \times 6)$

Since 6 is constant: Time Complexity = $O(D \times S)$

Space Complexity = $O(D \times S)$

LPS:

For a string of length n: Number of DP states = n^2

Each state takes constant time.

Time Complexity = $O(n^2)$ Space Complexity = $O(n^2)$

For "ABCBA", n = 5, so the DP table contains $5 \times 5 = 25$ possible states.

E. Conclusion

Both Dice Throw and LPS use Dynamic Programming, but their DP structures are different. Dice Throw uses a counting state based on the number of dice and the target sum, whereas

LPS uses an interval state based on two string indices. For 2 dice and sum 5, there are 4 possible solutions. For the string "ABCBA", the longest palindromic subsequence is "ABCBA" with length 5.

3.BELLMAN-FORD VS FLOYD (ITERATION VS MATRIX DP)

A. Given Graph

The graph contains the following weighted edges:

• A → B = 6
• A → C = 5
• B → D = 1
• C → D = 2

Source vertex = A

The objective is to find the shortest paths from A to all other vertices.

B. Bellman-Ford Algorithm

Bellman-Ford finds shortest paths by repeatedly relaxing all edges.

Initial distances from A:

Vertex	A	B	C	D
Initial	0	∞	∞	∞

There are 4 vertices, so Bellman-Ford performs at most: V − 1 = 4 − 1 = 3 iterations

Iteration 1:

1. Relax A → B (6): B = 0 + 6 = 6
2. Relax A → C (5): C = 0 + 5 = 5
3. Relax B → D (1): D = 6 + 1 = 7
4. Relax C → D (2): D = min(7, 5 + 2) = 7

After Iteration 1:

Vertex	A	B	C	D
Distance	0	6	5	7

Iteration 2: No distance can be improved.

Vertex	A	B	C	D
Distance	0	6	5	7

Iteration 3: Again, no updates occur.

Vertex	A	B	C	D
Distance	0	6	5	7

Therefore, the shortest paths are:

• A → A = 0 A → B = 6 A → C = 5 A → C → D = 7

Shortest distance from A to D = 7.

C. Floyd-Warshall Algorithm

Floyd-Warshall calculates shortest paths between every pair of vertices using a distance matrix.

Initial matrix:

From / To	**A**	**B**	**C**	**D**
A	0	6	5	∞
B	∞	0	∞	1
C	∞	∞	0	2
D	∞	∞	∞	0

The recurrence is: D[i][j] = min(D[i][j], D[i][k] + D[k][j])

where k is used as an intermediate vertex.

D. Floyd-Warshall Updates

Using B as an intermediate vertex: A → B → D A → D = min(∞, 6 + 1) A → D = 7

Updated matrix:

From / To	A	B	C	D
A	0	6	5	7
B	∞	0	∞	1
C	∞	∞	0	2
D	∞	∞	∞	0

Using C as an intermediate vertex: $A \rightarrow C \rightarrow D$ $A \rightarrow D = \min(7, 5 + 2)$ $A \rightarrow D = 7$

There is no further improvement.

Final Floyd-Warshall matrix:

From / To	**A**	**B**	**C**	**D**
A	0	6	5	7
B	∞	0	∞	1
C	∞	∞	0	2
D	∞	∞	∞	0

E. Comparison of Updates

Feature	**Bellman-Ford**	**Floyd-Warshall**
Approach	Repeated edge relaxation	Matrix-based DP
Purpose	Single-source shortest path	All-pairs shortest path
State	Distance from source	Distance between every pair
Main operation	Relax each edge	Check intermediate vertex
Recurrence	d[v] = min(d[v], d[u] + w)	D[i][j] = min(D[i][j], D[i][k] + D[k][j])

Result from A to D	7	7
Negative edges	Supported	Supported
Negative cycle detection	Yes	Yes

F. Efficiency Analysis

Bellman-Ford:

For V vertices and E edges:

Time Complexity = $O(V \times E)$ Space Complexity = $O(V)$

For this graph:

$V = 4$ $E = 4$

Time = $O(4 \times 4) = O(16)$

Floyd-Warshall:

For V vertices: Time Complexity = $O(V^3)$ Space Complexity = $O(V^2)$

For this graph:

$V = 4$

Time = $O(4^3) = O(64)$

G. Conclusion

Both algorithms produce the same shortest-path results: $A \rightarrow B = 6$ $A \rightarrow C = 5$ $A \rightarrow D = 7$

The shortest path from A to D is: $A \rightarrow C \rightarrow D = 5 + 2 = 7$

Bellman-Ford uses repeated edge relaxation and is more suitable for finding shortest paths from a single source. Floyd-Warshall uses a matrix-based dynamic programming approach and is more suitable when shortest paths between all pairs of vertices are required. For a sparse graph with a single source, Bellman-Ford is generally more efficient, while Floyd-Warshall is convenient for all-pairs shortest paths.